

1. How you framed the problem — which errors matter more and why.

Answer

I framed this as a risk-ranking problem, not just a binary classification problem. The business constraint is that the review team can only inspect the top 25% of claims before submission, so the most important question is: how many actual denials can we capture inside that review capacity?

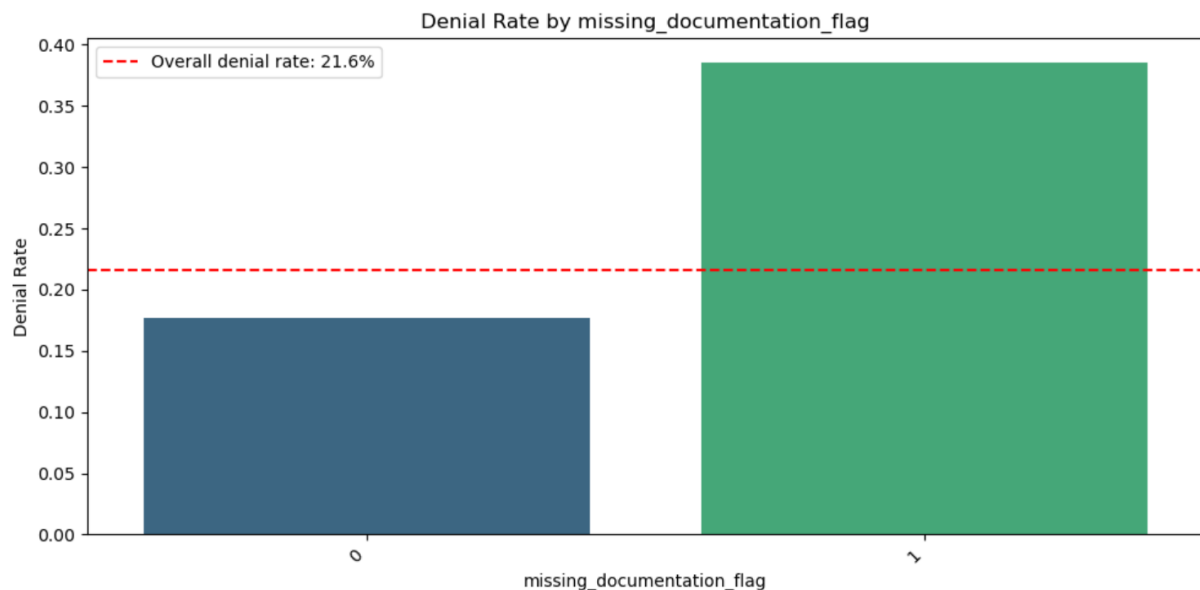
The idea was to put a model in place that can capture the maximum number of actual denials in top 25% claims.

False negatives matter most because a denied claim that is not flagged goes out without review and creates downstream rework, delays, and cost. False positives still matter because analyst capacity is limited, but reviewing a clean claim is less costly than missing a preventable denial. Because of that, I prioritized denial capture in the top 25% of risk scores, with PR-AUC and ROC-AUC as supporting metrics.

2. Two or three data findings worth sharing with a non-technical manager.

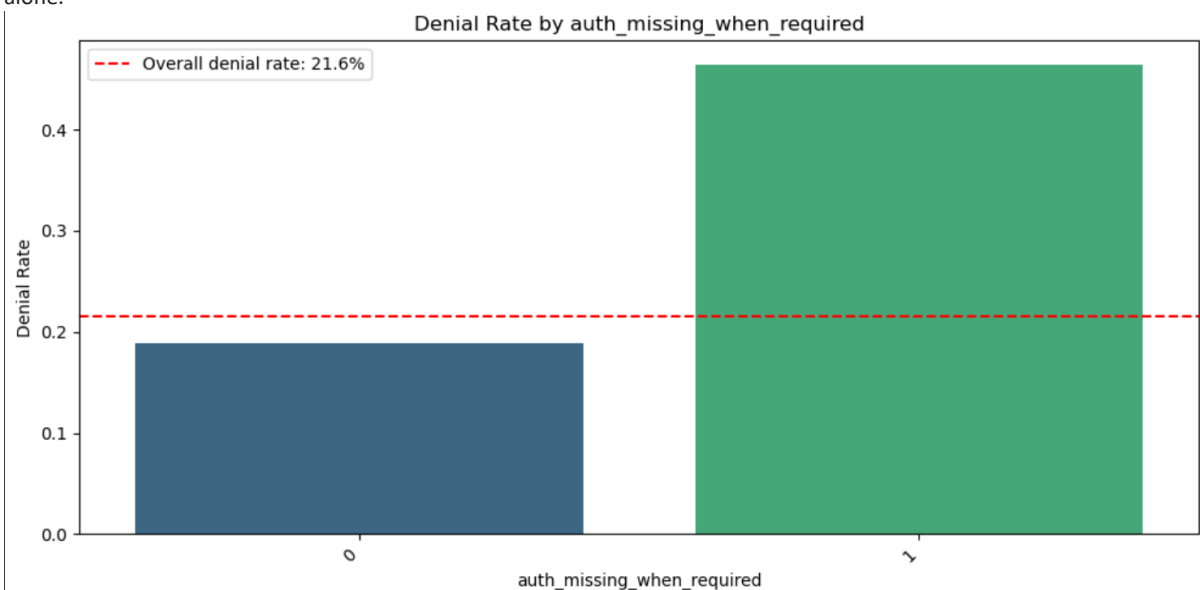
a. Missing documentation is one of the strongest risk signals.

Claims flagged for missing documentation deny at about 38.6% versus 17.6% when documentation is not flagged.



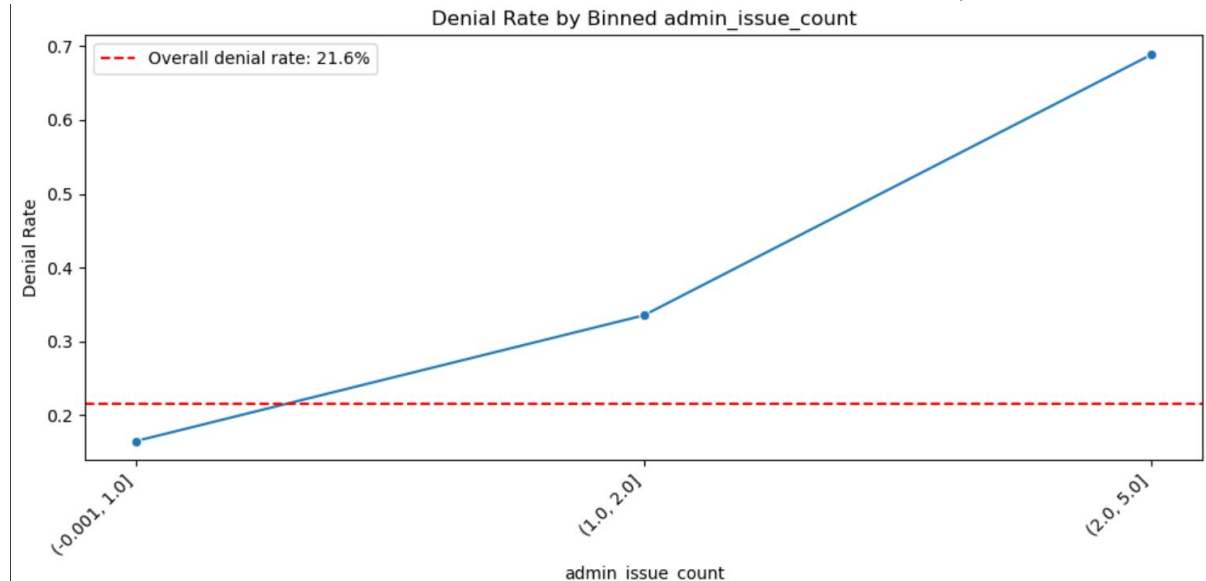
b. Authorization gaps matter more when authorization is actually required.

Authorization missing when required denies at about 46.5%, much stronger than looking at prior authorization status alone.



c. Claims with multiple administrative issues are much riskier.

Denial rate rises from about 12% with zero admin issues to more than 66% when three issues are present.



3. What you built, what baselines you compared against, and your threshold choice

Answer

- **What I built?**
Used the provided train / validation / test split exactly as given; no custom split was created.
- Excluded leakage fields: claim_id, split, is_denied, and denial_reason.
- Engineered interpretable features such as auth_missing_when_required, referral_missing_when_required, payment_to_billed_ratio, late_submission_30d, complex_claim, admin_issue_count, and expected_payment_tier.
- Compared Logistic Regression, Random Forest, Histogram Gradient Boosting, and XGBoost with randomized parameter search.
- The final recommended model was tuned Logistic Regression, because it had strong validation performance and generalized better than the tree-based models, which showed more overfitting.

Threshold Choice

The operational threshold is the validation-set 75th percentile of predicted denial probability. This directly matches the business constraint: only the top 25% highest-risk claims should be routed for manual review.

4. Test set metrics and denial capture at top 25%.

ROC-AUC	0.6902
PR-AUC	0.5098
Precision / Recall / F1	0.4714 / 0.4714 / 0.4714
Threshold	0.2764
Claims reviewed	135 claims, representing the top 25% of the test set
Denials captured	65 of 140 actual denials
Denial capture rate at top 25%	46.43%
Precision in reviewed claims	48.15%
Lift vs base denial rate	1.85x

5. Your LLM prompt, one example explanation, and one honest limitation.

Answer

Prompt Design

Grounding	Use only supplied claim fields, risk score, and model risk factors; do not invent payer policy, clinical details, or missing context.
Reasoning flow	Internally identify supported risk factors, translate them into plain English, choose one action, and state uncertainty.
Output style	2-3 sentences, plain language, one recommended action, and a reminder that the score is a risk estimate.
Few-shot examples	Examples were drawn from high-risk current claims such as CCLM-00082, CCLM-00177, and CCLM-00087.

The Prompt:

prompt = f"""

You are helping a hospital claims analyst review denial risk before claim submission. Write a short, plain-English explanation for one claim.

Important rules: Use only the claim fields and risk factors provided. Do not invent facts, policies, clinical details, or missing context. Mention that the prediction is a risk estimate, not a guaranteed denial. Include one specific recommended action. Keep the final explanation to 2-3 sentences. Do not show step-by-step reasoning or chain-of-thought in the final answer.

Private reasoning flow: Identify the strongest supported risk factors, convert them into plain English, recommend one concrete pre-submission action, add uncertainty language, and output only the final explanation.

Few-shot examples:

Example 1

Input: Claim ID: CCLM-00082 | Predicted denial probability: 0.802 | Key risk factors: missing documentation, eligibility not verified, referral required but not present | Claim fields: payer_type: BCBS; visit_type: Inpatient; total_billed: 87589.75; expected_payment: 52699.88; prior_auth_required: 0; has_prior_auth: 0; is_in_network: 1; days_to_submit: 27; missing_documentation_flag: 1; eligibility_verified: 0; referral_required: 1; referral_present: 0

Output: This claim has an estimated denial risk of 80.2%, mainly because documentation is missing, eligibility was not verified, and a required referral is not present. This is a risk estimate, not a guarantee of denial; before submission, verify coverage and attach the missing referral or supporting documentation.

Example 2

Input: Claim ID: CCLM-00177 | Predicted denial probability: 0.634 | Key risk factors: missing documentation, prior authorization required but not on file, claim submitted 30 or more days after service | Claim fields: payer_type: Commercial; visit_type: Outpatient; total_billed: 8737.48; expected_payment: 6027.84; prior_auth_required: 1; has_prior_auth: 0; is_in_network: 1; days_to_submit: 34; missing_documentation_flag: 1; eligibility_verified: 1; referral_required: 0; referral_present: 0

Output: This claim has an estimated denial risk of 63.4%, mainly because documentation is missing, prior authorization is required but not on file, and the claim was submitted 34 days after service. This is a risk estimate, not a guarantee of denial; before submission, confirm authorization and review timely filing requirements.

Now write the explanation for this claim.

Input: Claim ID: {claim_row["claim_id"]} | Predicted denial probability: {denial_probability:.3f} | Key risk factors: {"", ".join(top_risk_factors)} | Claim fields: payer_id: {claim_row.get("payer_id")}; payer_type: {claim_row.get("payer_type")}; visit_type: {claim_row.get("visit_type")}; total_billed: {claim_row.get("total_billed")}; expected_payment: {claim_row.get("expected_payment")}; num_procedures: {claim_row.get("num_procedures")}; num_diagnoses: {claim_row.get("num_diagnoses")}; prior_auth_required: {claim_row.get("prior_auth_required")}; has_prior_auth: {claim_row.get("has_prior_auth")}; is_in_network: {claim_row.get("is_in_network")}; days_to_submit: {claim_row.get("days_to_submit")}; missing_documentation_flag: {claim_row.get("missing_documentation_flag")}; eligibility_verified: {claim_row.get("eligibility_verified")}; referral_required: {claim_row.get("referral_required")}; referral_present: {claim_row.get("referral_present")}; service_month: {claim_row.get("service_month")}

Output:

""""

one example explanation

This claim has an estimated denial risk of 80.2%, mainly because documentation is missing, eligibility was not verified, and a required referral is not present. This is a risk estimate, not a guarantee of denial; before submission, verify coverage and attach the missing referral or supporting documentation.

Limitation

The explanations are based on available structured fields and model risk drivers, not on claim notes, payer policy text, or detailed clinical documentation. In a real deployment, those additional sources could materially improve both risk prediction and recommended actions.

6. What you would improve with more time.

- a. Calibrate probabilities so the denial scores better reflect true likelihood, not just ranking.
- b. Add SHAP or another local explanation method to produce more model-faithful claim-level risk drivers.
- c. Test payer-specific models or payer-specific thresholds, since denial behavior varies across payer types.
- d. Add historical claim volume and provider-level features if available.
- e. Evaluate operational outcomes, such as analyst workload, preventable denial rate, etc